CPSC 304 Project Cover Page

Milestone #: 2

Date: Feb 26 2026

Group Number: 113

Name	Student Number	CS Alias (Userid)	Preferred E-mail Address
Joseph Song	38031209	q1u8s	jsong1618@gmail.com
Ali Javed	24599169	v3w0b	alijavedansari@gmail.com
Runju Huang	38225363	t9g1z	runjuuuuu@gmail.com

By typing our names and student numbers in the above table, we certify that the work in the attached assignment was performed solely by those whose names and student IDs are included above.

In addition, we indicate that we are fully aware of the rules and consequences of plagiarism, as set forth by the Department of Computer Science and the University of British Columbia

2. Summary of the project

This app helps beginners save money by tracking their groceries. It uses AI to suggest recipes based on what's in their kitchen, and users can share their inventory across multiple accounts. Users can also share their successful meals so the community can find recipes that actually work.

3. Project Stack

- Frontend: TypeScript, React
- Backend: TypeScript, Nest.js with express, PostgreSQL 17

4. ER diagram

(The diagram is on the next page)

Changes made after Milestone 1:

- PaidAccount entity
    - SubscriptionPriceInCents was added to meet the non-PK/CK FD requirement, assuming the price is fixed
- React relationship between Post and Account entities
    - Kind was added to meet the non-PK/CK FDs requirement
- Ingredient entity
    - Fat was added to replace calories, since we can calculate calories from carbs, protein, and fat using a formula:

        Total Calories = (Grams of Protein * 4) + (Grams of Carbs * 4) + (Grams of Fat * 9)
- Recipe entity
    - Content was added because we forgot to include it in Milestone 1
    - Rename Cost to CostInCents to make it clear that we store the cents

5. The schema derived from ER diagram

Note:

- PKs stand for primary keys
- CKs stand for candidate keys
- FKs stand for foreign keys
- FDs stand for functional dependencies
- PostgreSQL supports UUID, Timestamp, and Enum types, so we used them directly

Account(<u>AccountID</u>: UUID, Email: String, Username: String, Password: String)

- PKs: {AccountID}
- (non-PK) CKs: {Email}, {Username}
- Not null: Email, Username, Password
- ISA: TrialAccount and PaidAccount (Total/Disjoint)

TrialAccount(**<u>AccountID</u>**: UUID, UsageRemaining: Int, TrialStartDate: Timestamp, TrialEndDate: Timestamp)

- PKs: {AccountID}
- (non-PK) CKs: none
- FKs: {AccountID} references Account(AccountID)
- Not null: UsageRemaining, TrialStartDate, TrialEndDate

PaidAccount(

<u>AccountID</u>: UUID,
SubscriptionStartDate: Timestamp,
SubscriptionEndDate: Timestamp,
SubscriptionType: Enum('monthly', 'yearly'),
SubscriptionPriceInCents: Int

)

- PKs: {AccountID}
- (non-PK) CKs: none
- FKs: {AccountID} references Account(AccountID)
- Not null: SubscriptionStartDate, SubscriptionEndDate, SubscriptionType, SubscriptionPriceInCents
- Note: SubscriptionPriceInCents was added to meet the non-PK/CK FD requirement, assuming the price is fixed

Post(

<u>PostID</u>: UUID,
Title: String,
Body: String,
CreationDate: Timestamp,
Visibility: Enum('private', 'public'),
AccountID: UUID,
RecipeID: UUID

)

- PKs: {PostID}
- (non-PK) CKs: none
- FK
  - {AccountID} references Account(AccountID)
  - {RecipeID} references Recipe(RecipeID)

- Not null: Title, Body, CreationDate, Visibility, AccountID
- Note: RecipeID might be null if the user doesn't link the post with a recipe

PostReaction(**<u>PostID</u>**: UUID, **<u>AccountID</u>**: UUID, Emoji: String, Kind: Enum('Positive', 'Negative', 'Neutral'))
- PKs: {PostID, AccountID}
- (non-PK) CKs: none
- FK
    - {AccountID} references Account(AccountID)
    - {PostID} references Post(PostID)
- Not null: Emoji, Kind
- Note:
    - Emoji should contain only one emoji character
    - Kind is added to meet the non-PK/CK FDs requirement

Ingredient(<u>Name</u>: String, Carbs: Float, Protein: Float, Fat: Float)
- PKs: {Name}
- (non-PK) CKs: none
- Not null: Carbs, Protein, Fat
- Note: Fat is a newly added attribute that replaces Calories, since we can calculate Calories from Carbs, Protein, and Fat

Recipe(
    <u>RecipeID</u>: UUID,
    Name: String,
    Content: String,
    CostInCents: Int,
    Time: Int,
    Cuisine: String,
    CreationDate: Timestamp,
    ModificationDate: Timestamp,
    AccountID: UUID
)
- PKs: {RecipeID}
- (non-PK) CKs: none
- FKs: {AccountID} references Account(AccountID)
- Not null: AccountID, Name, Content, CostInCents, Time, CreationDate, ModificationDate
- Note:
    - Time represent total seconds
    - Content is a newly added attribute (we forgot to add it in Milestone 1's ER diagram)
    - Rename Cost to CostInCents to make it clear that we store the cents
    - Cuisine might be null

RecipeIngredient(**<u>RecipeID</u>**: UUID, **<u>IngredientName</u>**: String, Quantity: Int)
- PKs: {RecipeID, IngredientName}
- (non-PK) CKs: none
- FKs:
    - {RecipeID} references Recipe(RecipeID)
    - {IngredientName} references Ingredient(Name)
- Not null: Quantity

- Note: We couldn't force Recipe's total participation constraints until we learn assertions in the SQL unit

RecipeNote(
    <u>RecipeNoteID</u>: UUID,
    Photo: String,
    Note: String,
    CreationDate: Timestamp,
    ModificationDate: Timestamp,
    AccountID: UUID
)
- PKs: {RecipeNoteID}
- (non-PK) CKs: none
- FKs: {AccountID} references Account(AccountID)
- Not null: AccountID, Note, CreationDate, ModificationDate
- Note: Photo is a file path and might be null if the user didn't upload an image

RecipeNoteAnnotatesRecipe(**<u>RecipeNoteID</u>**: UUID, **<u>RecipeID</u>**: UUID)
- PKs: {RecipeNoteID, RecipeID}
- (non-PK) CKs: none
- FKs:
    - {RecipeNoteID} references RecipeNote(RecipeNoteID)
    - {RecipeID} references Recipe(RecipeID)

Receipt(
    <u>ReceiptID</u>: UUID,
    OCRResult: String,
    ImageFilePath: String,
    CreationDate: Timestamp,
    AccountID: UUID
)
- PKs: {ReceiptID}
- (non-PK) CKs: {ImageFilePath}
- FKs: {AccountID} references Account(AccountID)
- Not null: ImageFilePath, CreationDate, AccountID
- Note: OCRResult might be null, as it depends on the results from external APIs

Inventory(<u>InventoryID</u>: UUID, Name: String, Description: String, Type: String)
- PKs: {InventoryID}
- (non-PK) CKs: none
- Not null: Name, Description, Type

InventoryItem(
    <u>InventoryItemID</u>: Int,
    <u>InventoryID</u>: UUID,
    Quantity: Int,
    CreationDate: Timestamp,
    ExpirationDate: Timestamp,
    IngredientName: String,
    ReceiptID: UUID

)

- PKs: {InventoryItemID, InventoryID}
- (non-PK) CKs: none
- FKs:
    - {InventoryID} references Inventory(InventoryID)
    - {IngredientName} references Ingredient(Name)
    - {ReceiptID} references Receipt(ReceiptID)
- Not null: IngredientName, Quantity, CreationDate
- Note:
    - ExpirationDate might be null if the user chooses not to have one
    - ReceiptID might be null if the user adds an item manually without uploading a receipt

AccountOwnsInventory(**<u>AccountID</u>**: UUID, **<u>InventoryID</u>**: UUID)

- PKs: {AccountID, InventoryID}
- (non-PK) CKs: none
- FKs:
    - {AccountID} references Account(AccountID)
    - {InventoryID} references Inventory(InventoryID)
- Note: We couldn't force Inventory's total participation constraints in many-to-many relationships until we learn assertions in the SQL unit

6. Functional Dependencies

Account(<u>AccountID</u>, Email, Username, Password)

- AccountID -> Email, Username, Password
- Email -> AccountID, Username, Password
- Username -> AccountID, Email, Password

TrialAccount(**<u>AccountID</u>**, UsageRemaining, TrialStartDate, TrialEndDate)

- AccountID -> UsageRemaining, TrialStartDate, TrialEndDate

PaidAccount(**<u>AccountID</u>**, SubscriptionStartDate, SubscriptionEndDate, SubscriptionType, SubscriptionPriceInCents)

- AccountID -> SubscriptionStartDate, SubscriptionEndDate, SubscriptionType, SubscriptionPriceInCents
- SubscriptionType -> SubscriptionPriceInCents

Post(<u>PostID</u>, Title, Body, CreationDate, Visibility, **AccountID**, **RecipeID**)

- PostID -> Title, Body, CreationDate, Visibility, AccountID, RecipeID

PostReaction(**<u>PostID</u>**, **<u>AccountID</u>**, Emoji, Kind)

- PostID, AccountID -> Emoji, Kind
- Emoji -> Kind

Ingredient(<u>Name</u>, Carbs, Protein, Fat)

- Name -> Carbs, Protein, Fat

Recipe(<u>RecipeID</u>, Name, Content, CostInCents, Time, Cuisine, CreationDate, ModificationDate, **AccountID**)

- RecipeID -> Name, Content, CostInCents, Time, Cuisine, CreationDate, ModificationDate, AccountID

RecipeIngredient(**RecipeID**, **IngredientName**, Quantity)
- RecipeID, IngredientName -> Quantity

RecipeNote(RecipeNoteID, Photo, Note, CreationDate, ModificationDate, **AccountID**)
- RecipeNoteID -> Photo, Note, CreationDate, ModificationDate, AccountID

RecipeNoteAnnotatesRecipe(**RecipeNoteID**, **RecipeID**)
- No non-trivial FD from RecipeNoteAnnotatesRecipe

Receipt(ReceiptID, OCRResult, ImageFilePath, CreationDate, **AccountID**)
- ReceiptID -> OCRResult, ImageFilePath, CreationDate, AccountID
- ImageFilePath -> ReceiptID, OCRResult, CreationDate, AccountID

Inventory(InventoryID, Name, Description, Type)
- InventoryID -> Name, Description, Type

InventoryItem(
    InventoryItemID,
    InventoryID,
    Quantity,
    CreationDate,
    ExpirationDate,
    IngredientName,
    ReceiptID
)
- InventoryItemID, InventoryID -> Quantity, CreationDate, ExpirationDate, IngredientName, ReceiptID

AccountOwnsInventory(**AccountID**, **InventoryID**)
- No non-trivial FD from AccountOwnsInventory

7.1 Intermediate Normalization Steps

Account(AccountID: UUID, Email: String, Username: String, Password: String)
- FDs
    - AccountID -> Email, Username, Password
    - Email -> AccountID, Username, Password
    - Username -> AccountID, Email, Password
- PKs: {AccountID}
- (non-PK) CKs: {Email}, {Username}
- Note: Since {AccountID} is PK, and {Email} and {Username} are CKs, Account is already in BCNF because PKs and CKs are automatically superkeys

TrialAccount(**AccountID**: UUID, UsageRemaining: Int, TrialStartDate: Timestamp, TrialEndDate: Timestamp)
- FDs
    - AccountID -> UsageRemaining, TrialStartDate, TrialEndDate

- PKs: {AccountID}
- (non-PK) CKs: none
- FKs: {AccountID} references Account(AccountID)
- Note: Since {AccountID} is PK, TrialAccount is already in BCNF because PKs and CKs are automatically superkeys

PaidAccount(
    <u>AccountID</u>: UUID,
    SubscriptionStartDate: Timestamp,
    SubscriptionEndDate: Timestamp,
    SubscriptionType: Enum('monthly', 'yearly'),
    SubscriptionPriceInCents: Int
)

- FDs
    - AccountID -> SubscriptionStartDate, SubscriptionEndDate, SubscriptionType, SubscriptionPriceInCents
    - SubscriptionType -> SubscriptionPriceInCents
- Normalization
    - SubscriptionType -> SubscriptionPriceInCents violate both BCNF and 3NF since SubscriptionType is not a superkey and SubscriptionPriceInCents is not part of a key
    - Minimum cover:
        - Step 1, Put FDs in standard form
            - AccountID -> SubscriptionStartDate
            - AccountID -> SubscriptionEndDate
            - AccountID -> SubscriptionType
            - AccountID -> SubscriptionPriceInCents
            - SubscriptionType -> SubscriptionPriceInCents
        - Step 2, Minimize LHS of each FD
            - Nothing needs to be minimized at this step
        - Step 3, Delete Redundant FDs
            - Delete AccountID -> SubscriptionPriceInCents since we can recover it from AccountID -> SubscriptionType and SubscriptionType -> SubscriptionPriceInCents
        - Final result
            - AccountID -> SubscriptionStartDate
            - AccountID -> SubscriptionEndDate
            - AccountID -> SubscriptionType
            - SubscriptionType -> SubscriptionPriceInCents
    - Decomposition into 3NF using lossless join method
        - Step 1, decompose into BCNF using minimum cover
            - SubscriptionType -> SubscriptionPriceInCents violate BCNF since SubscriptionType is not a superkey
                - R1(AccountID, SubscriptionStartDate, SubscriptionEndDate, SubscriptionType)
                - R2(SubscriptionType, SubscriptionPriceInCents)
            - R1 and R2 are now in BCNF
        - Step 2, identify the set of dependencies in minimum cover that are not preserved by the decomposition
            - Every FD is preserved

- Step 3, for each X -> b in the last step, create a relation $R_n(X \cup b)$ and add it to the decomposition
    - Nothing to add
- Final result (rename R1 to PaidAccount and R2 to Subscription for better semantics)
    - PaidAccount(
        <u>AccountID</u>: UUID,
        SubscriptionStartDate: Timestamp,
        SubscriptionEndDate: Timestamp,
        SubscriptionType: Enum('monthly', 'yearly')
)
        - PKs: {AccountID}
        - (non-PK) CKs: none
        - FKs:
            - {AccountID} references Account(AccountID)
            - {SubscriptionType} references Subscription(SubscriptionType)
    - Subscription(<u>SubscriptionType</u>: Enum('monthly', 'yearly'), SubscriptionPriceInCents: Int)
        - PKs: {SubscriptionType}
        - (non-PK) CKs: none

Post(
    <u>PostID</u>: UUID,
    Title: String,
    Body: String,
    CreationDate: Timestamp,
    Visibility: Enum('private', 'public'),
    AccountID: UUID,
    RecipeID: UUID
)

- FDs
    - PostID -> Title, Body, CreationDate, Visibility, AccountID, RecipeID
- PKs: {PostID}
- (non-PK) CKs: none
- FK
    - {AccountID} references Account(AccountID)
    - {RecipeID} references Recipe(RecipeID)
- Note: Since {PostID} is PK, Post is already in BCNF because PKs and CKs are automatically superkeys

PostReaction(**<u>PostID</u>**: UUID, **<u>AccountID</u>**: UUID, Emoji: String, Kind: Enum('Positive', 'Negative', 'Neutral'))

- FDs
    - PostID, AccountID -> Emoji, Kind
    - Emoji -> Kind
- Normalization
    - Emoji -> Kind violate both BCNF and 3NF since Emoji is not a superkey and Kind is not part of a key
    - Minimum cover:
        - Step 1, Put FDs in standard form
            - PostID, AccountID -> Emoji

            - PostID, AccountID -> Kind
            - Emoji -> Kind
        - Step 2, Minimize LHS of each FD
            - $PostID^+$ = {PostID}, $AccountID^+$ = {AccountID}
            - Since Emoji is not in either $PostID^+$ or $AccountID^+$, neither attribute can be removed
        - Step 3, Delete Redundant FDs
            - Delete PostID, AccountID -> Kind since we can recover it from PostID, AccountID -> Emoji and Emoji -> Kind
        - Final result
            - PostID, AccountID -> Emoji
            - Emoji -> Kind
    - Decomposition into 3NF using synthesis method
        - Step 1, For each FD X->b in minimum cover Add relation Xb to the decomposition
            - R1(PostID, AccountID, Emoji)
            - R2(Emoji, Kind)
        - Step 2, Check if the decomposition includes all the attributes of a key
            - Yes, R1 includes all the attributes of a key
        - Step 3, Check redundant relations
            - Nothing to remove
        - Final result (rename R1 to PostReaction and R2 to EmojiMeaning for better semantics)
            - PostReaction(**<u>PostID</u>**: UUID, **<u>AccountID</u>**: UUID, **Emoji**: String)
                - PKs: {PostID, AccountID}
                - (non-PK) CKs: none
                - FK
                    - {AccountID} references Account(AccountID)
                    - {PostID} references Post(PostID)
                    - {Emoji} references EmojiMeaning(Emoji)
            - EmojiMeaning(<u>Emoji</u>: String, Kind: Enum('Positive', 'Negative', 'Neutral'))
                - PKs: {Emoji}
                - (non-PK) CKs: none

Ingredient(<u>Name</u>: String, Carbs: Float, Protein: Float, Fat: Float)
- FDs
    - Name -> Carbs, Protein, Fat
- PKs: {Name}
- (non-PK) CKs: none
- Note: Since {Name} is PK, Ingredient is already in BCNF because PKs and CKs are automatically superkeys

Recipe(
    <u>RecipeID</u>: UUID,
    Name: String,
    Content: String,
    CostInCents: Int,
    Time: Int,
    Cuisine: String,
    CreationDate: Timestamp,
    ModificationDate: Timestamp,

AccountID: UUID
)

- FDs
  - RecipeID -> Name, Content, CostInCents, Time, Cuisine, CreationDate, ModificationDate, AccountID
- PKs: {RecipeID}
- (non-PK) CKs: none
- FKs: {AccountID} references Account(AccountID)
- Note: Since {RecipeID} is PK, Recipe is already in BCNF because PKs and CKs are automatically superkeys

RecipeIngredient(**<u>RecipeID</u>**: UUID, **<u>IngredientName</u>**: String, Quantity: Int)

- FDs
  - RecipeID, IngredientName -> Quantity
- PKs: {RecipeID, IngredientName}
- (non-PK) CKs: none
- FKs:
  - {RecipeID} references Recipe(RecipeID)
  - {IngredientName} references Ingredient(Name)
- Note: Since {RecipeID, IngredientName} is PK, RecipeIngredient is already in BCNF because PKs and CKs are automatically superkeys

RecipeNote(
<u>RecipeNoteID</u>: UUID,
Photo: String,
Note: String,
CreationDate: Timestamp,
ModificationDate: Timestamp,
AccountID: UUID
)

- FDs
  - RecipeNoteID -> Photo, Note, CreationDate, ModificationDate, AccountID
- PKs: {RecipeNoteID}
- (non-PK) CKs: none
- FKs: {AccountID} references Account(AccountID)
- Note: Since {RecipeNoteID} is PK, RecipeNote is already in BCNF because PKs and CKs are automatically superkeys

RecipeNoteAnnotatesRecipe(**<u>RecipeNoteID</u>**: UUID, **<u>RecipeID</u>**: UUID)

- PKs: {RecipeNoteID, RecipeID}
- (non-PK) CKs: none
- FKs:
  - {RecipeNoteID} references RecipeNote(RecipeNoteID)
  - {RecipeID} references Recipe(RecipeID)
- No non-trivial FD from RecipeNoteAnnotatesRecipe

Receipt(
<u>ReceiptID</u>: UUID,
OCRResult: String,
ImageFilePath: String,

CreationDate: Timestamp,
AccountID: UUID
)

- FDs
  - ReceiptID -> OCRResult, ImageFilePath, CreationDate, AccountID
  - ImageFilePath -> ReceiptID, OCRResult, CreationDate, AccountID
- PKs: {ReceiptID}
- (non-PK) CKs: {ImageFilePath}
- FKs: {AccountID} references Account(AccountID)
- Note: Since {ReceiptID} is PK and {ImageFilePath} is CK, Receipt is already in BCNF because PKs and CKs are automatically superkeys

Inventory(<u>InventoryID</u>: UUID, Name: String, Description: String, Type: String)

- FDs
  - InventoryID -> Name, Description, Type
- PKs: {InventoryID}
- (non-PK) CKs: none
- Note: Since {InventoryID} is PK, Inventory is already in BCNF because PKs and CKs are automatically superkeys

InventoryItem(
<u>InventoryItemID</u>: Int,
<u>InventoryID</u>: UUID,
Quantity: Int,
CreationDate: Timestamp,
ExpirationDate: Timestamp,
IngredientName: String,
ReceiptID: UUID
)

- FDs
  - InventoryItemID, InventoryID -> Quantity, CreationDate, ExpirationDate, IngredientName, ReceiptID
- PKs: {InventoryItemID, InventoryID}
- (non-PK) CKs: none
- FKs:
  - {InventoryID} references Inventory(InventoryID)
  - {IngredientName} references Ingredient(Name)
  - {ReceiptID} references Receipt(ReceiptID)
- Note: Since {InventoryItemID, InventoryID} is PK, InventoryItem is already in BCNF because PKs and CKs are automatically superkeys

AccountOwnsInventory(**<u>AccountID</u>**: UUID, **<u>InventoryID</u>**: UUID)

- PKs: {AccountID, InventoryID}
- (non-PK) CKs: none
- FKs:
  - {AccountID} references Account(AccountID)
  - {InventoryID} references Inventory(InventoryID)
- No non-trivial FD from AccountOwnsInventory

7.2 Final Normalized Schema

Account(<u>AccountID</u>: UUID, Email: String, Username: String, Password: String)
- PKs: {AccountID}
- (non-PK) CKs: {Email}, {Username}
- Not null: Email, Username, Password
- ISA: TrialAccount and PaidAccount (Total/Disjoint)

TrialAccount(**<u>AccountID</u>**: UUID, UsageRemaining: Int, TrialStartDate: Timestamp, TrialEndDate: Timestamp)
- PKs: {AccountID}
- (non-PK) CKs: none
- FKs: {AccountID} references Account(AccountID)
- Not null: UsageRemaining, TrialStartDate, TrialEndDate

PaidAccount(
    <u>AccountID</u>: UUID,
    SubscriptionStartDate: Timestamp,
    SubscriptionEndDate: Timestamp,
    SubscriptionType: Enum('monthly', 'yearly')
)
- PKs: {AccountID}
- (non-PK) CKs: none
- FKs:
    - {AccountID} references Account(AccountID)
    - {SubscriptionType} references Subscription(SubscriptionType)
- Not null: SubscriptionStartDate, SubscriptionEndDate, SubscriptionType

Subscription(<u>SubscriptionType</u>: Enum('monthly', 'yearly'), SubscriptionPriceInCents: Int)
- PKs: {SubscriptionType}
- (non-PK) CKs: none
- Not null: SubscriptionPriceInCents
- Note: SubscriptionPriceInCents is in cents

Post(
    <u>PostID</u>: UUID,
    Title: String,
    Body: String,
    CreationDate: Timestamp,
    Visibility: Enum('private', 'public'),
    AccountID: UUID,
    RecipeID: UUID
)
- PKs: {PostID}
- (non-PK) CKs: none
- FK
    - {AccountID} references Account(AccountID)
    - {RecipeID} references Recipe(RecipeID)
- Not null: Title, Body, CreationDate, Visibility, AccountID
- Note: RecipeID might be null if the user doesn't link the post with a recipe

PostReaction(**<u>PostID</u>**: UUID, **<u>AccountID</u>**: UUID, **Emoji**: String)

- PKs: {PostID, AccountID}
- (non-PK) CKs: none
- FK
    - {AccountID} references Account(AccountID)
    - {PostID} references Post(PostID)
    - {Emoji} references EmojiMeaning(Emoji)
- Not null: Emoji
- Note: Emoji should contain only one emoji character

EmojiMeaning(<u>Emoji</u>: String, Kind: Enum('Positive', 'Negative', 'Neutral'))

- PKs: {Emoji}
- (non-PK) CKs: none
- Not null: Kind
- Note: Emoji should contain only one emoji character

Ingredient(<u>Name</u>: String, Carbs: Float, Protein: Float, Fat: Float)

- PKs: {Name}
- (non-PK) CKs: none
- Not null: Carbs, Protein, Fat

Recipe(

<u>RecipeID</u>: UUID,
Name: String,
Content: String,
CostInCents: Int,
Time: Int,
Cuisine: String,
CreationDate: Timestamp,
ModificationDate: Timestamp,
AccountID: UUID

)

- PKs: {RecipeID}
- (non-PK) CKs: none
- FKs: {AccountID} references Account(AccountID)
- Not null: AccountID, Name, Content, CostInCents, Time, CreationDate, ModificationDate
- Note:
    - Time represent total seconds
    - Cuisine might be null

RecipeIngredient(**<u>RecipeID</u>**: UUID, **<u>IngredientName</u>**: String, Quantity: Int)

- PKs: {RecipeID, IngredientName}
- (non-PK) CKs: none
- FKs:
    - {RecipeID} references Recipe(RecipeID)
    - {IngredientName} references Ingredient(Name)
- Not null: Quantity
- Note: We couldn't force Recipe's total participation constraints until we learn assertions in the SQL unit

RecipeNote(
    RecipeNoteID: UUID,
    Photo: String,
    Note: String,
    CreationDate: Timestamp,
    ModificationDate: Timestamp,
    AccountID: UUID
)

- PKs: {RecipeNoteID}
- (non-PK) CKs: none
- FKs: {AccountID} references Account(AccountID)
- Not null: AccountID, Note, CreationDate, ModificationDate
- Note: Photo is a file path and might be null if the user didn't upload an image

RecipeNoteAnnotatesRecipe(**RecipeNoteID**: UUID, **RecipeID**: UUID)

- PKs: {RecipeNoteID, RecipeID}
- (non-PK) CKs: none
- FKs:
    - {RecipeNoteID} references RecipeNote(RecipeNoteID)
    - {RecipeID} references Recipe(RecipeID)

Receipt(
    ReceiptID: UUID,
    OCRResult: String,
    ImageFilePath: String,
    CreationDate: Timestamp,
    AccountID: UUID
)

- PKs: {ReceiptID}
- (non-PK) CKs: {ImageFilePath}
- FKs: {AccountID} references Account(AccountID)
- Not null: ImageFilePath, CreationDate, AccountID
- Note: OCRResult might be null, as it depends on the results from external APIs

Inventory(InventoryID: UUID, Name: String, Description: String, Type: String)

- PKs: {InventoryID}
- (non-PK) CKs: none
- Not null: Name, Description, Type

InventoryItem(
    InventoryItemID: Int,
    InventoryID: UUID,
    Quantity: Int,
    CreationDate: Timestamp,
    ExpirationDate: Timestamp,
    IngredientName: String,
    ReceiptID: UUID
)

- PKs: {InventoryItemID, InventoryID}

- (non-PK) CKs: none
- FKs:
    - {InventoryID} references Inventory(InventoryID)
    - {IngredientName} references Ingredient(Name)
    - {ReceiptID} references Receipt(ReceiptID)
- Not null: IngredientName, Quantity, CreationDate
- Note:
    - ExpirationDate might be null if the user chooses not to have one
    - ReceiptID might be null if the user adds an item manually without uploading a receipt

AccountOwnsInventory(**AccountID**: UUID, **InventoryID**: UUID)
- PKs: {AccountID, InventoryID}
- (non-PK) CKs: none
- FKs:
    - {AccountID} references Account(AccountID)
    - {InventoryID} references Inventory(InventoryID)
- Note: We couldn't force Inventory's total participation constraints in many-to-many relationships until we learn assertions in the SQL unit

8. SQL DDL

```
CREATE TYPE Subscription_Type AS ENUM ('monthly', 'yearly');
CREATE TYPE Post_Visibility AS ENUM ('private', 'public');
CREATE TYPE Emoji_Kind AS ENUM ('Positive', 'Negative', 'Neutral');

CREATE TABLE Account(
	/* Note: Currently, we can't enforce the Total/Disjoint constraint until we learn the assertions. Another approach would be to remove the Account table and move everything to TrialAccount and PaidAccount, but that would require multiple FKs in other tables, which doesn't seem ideal. */
	AccountID UUID PRIMARY KEY,
	Email VARCHAR UNIQUE NOT NULL,
	Username VARCHAR UNIQUE NOT NULL,
	Password VARCHAR NOT NULL
);

CREATE TABLE TrialAccount(
	AccountID UUID PRIMARY KEY,
	/* Note: We might need to use assertions to ensure that UsageRemaining is not negative */
	UsageRemaining INT NOT NULL,
	TrialStartDate TIMESTAMPTZ NOT NULL,
	/* Note: We might need to use assertions to make sure the end date is after the start date */
	TrialEndDate TIMESTAMPTZ NOT NULL,
	FOREIGN KEY (AccountID) REFERENCES Account(AccountID)
		/* Use CASCADE here because If the account was deleted/updated, it should also be deleted/updated as well */
		ON DELETE CASCADE
		ON UPDATE CASCADE
```

```
);

CREATE TABLE Subscription(
	SubscriptionType Subscription_Type PRIMARY KEY,
	/* Note: We might need to use assertions to ensure that SubscriptionPriceInCents is not negative */
	SubscriptionPriceInCents INT NOT NULL
);

CREATE TABLE PaidAccount(
	AccountID UUID PRIMARY KEY,
	SubscriptionStartDate TIMESTAMPTZ NOT NULL,
	/* Note: We might need to use assertions to make sure the end date is after the start date */
	SubscriptionEndDate TIMESTAMPTZ NOT NULL,
	SubscriptionType Subscription_Type NOT NULL,
	FOREIGN KEY (AccountID) REFERENCES Account(AccountID)
		/* Use CASCADE here because If the account was deleted/updated, it should also be
deleted/updated as well */
		ON DELETE CASCADE
		ON UPDATE CASCADE,
	FOREIGN KEY (SubscriptionType) REFERENCES Subscription(SubscriptionType)
		/* Use NO ACTION here to prevent the Subscription from being deleted, because it doesn't
make sense to delete a subscription type if there's an account using it */
		ON DELETE NO ACTION
		/* Use CASCADE here because if the subscription updates, we should update it here too */
		ON UPDATE CASCADE
);

CREATE TABLE Recipe(
	RecipeID UUID PRIMARY KEY,
	Name VARCHAR NOT NULL,
	Content VARCHAR NOT NULL,
	CostInCents INT NOT NULL,
	Time INT NOT NULL,
	Cuisine VARCHAR,
	CreationDate TIMESTAMPTZ NOT NULL,
	ModificationDate TIMESTAMPTZ NOT NULL,
	AccountID UUID NOT NULL,
	FOREIGN KEY (AccountID) REFERENCES Account(AccountID)
		/* Use CASCADE here because if the account is deleted or updated, the recipe should also be
	deleted or updated. Otherwise, no one can see or need it, but it'll still be in our database */
		ON DELETE CASCADE
		ON UPDATE CASCADE
);

CREATE TABLE Post(
	PostID UUID PRIMARY KEY,
	Title VARCHAR NOT NULL,
	Body VARCHAR NOT NULL,
	CreationDate TIMESTAMPTZ NOT NULL,
```

```
	Visibility Post_Visibility NOT NULL,
	AccountID UUID NOT NULL,
	RecipeID UUID,
	FOREIGN KEY (AccountID) REFERENCES Account(AccountID)
		/* Use CASCADE here because if the account is deleted or updated, the post should also be
	deleted or updated. Otherwise, we'll end up with posts that aren't associated with any user. */
		ON DELETE CASCADE
		ON UPDATE CASCADE,
	FOREIGN KEY (RecipeID) REFERENCES Recipe(RecipeID)
		/* Use SET NULL here because if the recipe was deleted, the user's post can still exist, just
	won't connect to that recipe anymore */
		ON DELETE SET NULL
		/* Use CASCADE here because if the recipe is updated, we should update it here to keep the
	reference correct */
		ON UPDATE CASCADE
);

CREATE TABLE EmojiMeaning(
	Emoji VARCHAR PRIMARY KEY,
	Kind Emoji_Kind NOT NULL
);

CREATE TABLE PostReaction(
	PostID UUID,
	AccountID UUID,
	Emoji VARCHAR NOT NULL,
	PRIMARY KEY (PostID, AccountID),
	FOREIGN KEY (PostID) REFERENCES Post(PostID)
		/* Use CASCADE here because if the post is deleted or updated, the reaction should also be
deleted or updated. Otherwise, we'll end up with reactions that aren't associated with any post. */
		ON DELETE CASCADE
		ON UPDATE CASCADE,
	FOREIGN KEY (AccountID) REFERENCES Account(AccountID)
		/* Use CASCADE here because if the account is deleted or updated, the reaction should also be
deleted or updated. Otherwise, we'll end up with reactions that aren't associated with any user. */
		ON DELETE CASCADE
		ON UPDATE CASCADE,
	FOREIGN KEY (Emoji) REFERENCES EmojiMeaning(Emoji)
		/* Use NO ACTION here to prevent the EmojiMeaning from being deleted, because it doesn't
make sense to delete an emoji meaning if there's a reaction using it */
		ON DELETE NO ACTION
		/* Use CASCADE here because if the emoji meaning is updated, we should update it here to
	keep the reference correct */
		ON UPDATE CASCADE
);

CREATE TABLE Ingredient(
	Name VARCHAR PRIMARY KEY,
	Carbs NUMERIC(7,2) NOT NULL,
```

```
	Protein NUMERIC(7,2) NOT NULL,
	Fat NUMERIC(7,2) NOT NULL
);

/* Note: We couldn't force Recipe's total participation constraints until we learn assertions in the SQL unit */
CREATE TABLE RecipeIngredient(
	RecipeID UUID,
	IngredientName VARCHAR NOT NULL,
	/* Note: We might need to use assertions to ensure that Quantity is positive */
	Quantity INT NOT NULL,
	PRIMARY KEY (RecipeID, IngredientName),
	FOREIGN KEY (RecipeID) REFERENCES Recipe(RecipeID)
		/* Use CASCADE here because if the recipe is deleted or updated, the recipe ingredient should
	also be deleted or updated. Otherwise, we'll end up with recipe ingredients that aren't associated with
	any recipe. */
		ON DELETE CASCADE
		ON UPDATE CASCADE,
	FOREIGN KEY (IngredientName) REFERENCES Ingredient(Name)
		/* Use NO ACTION here to prevent the ingredient from being deleted, because it doesn't make
	sense to delete an ingredient if there's a recipe using it */
		ON DELETE NO ACTION
		/* Use CASCADE here because if the ingredient is updated, we should update it here to keep
	the reference correct */
		ON UPDATE CASCADE
);

CREATE TABLE RecipeNote(
	RecipeNoteID UUID PRIMARY KEY,
	Photo VARCHAR,
	Note VARCHAR NOT NULL,
	CreationDate TIMESTAMPTZ NOT NULL,
	ModificationDate TIMESTAMPTZ NOT NULL,
	AccountID UUID NOT NULL,
	FOREIGN KEY (AccountID) REFERENCES Account(AccountID)
		/* Use CASCADE here because if the account is deleted or updated, the recipe note should also
	be deleted or updated. Otherwise, we'll end up with recipe notes that aren't associated with any user. */
		ON DELETE CASCADE
		ON UPDATE CASCADE
);

CREATE TABLE RecipeNoteAnnotatesRecipe(
	RecipeNoteID UUID,
	RecipeID UUID,
	PRIMARY KEY (RecipeNoteID, RecipeID),
	FOREIGN KEY (RecipeNoteID) REFERENCES RecipeNote(RecipeNoteID)
		/* Use CASCADE here because if the recipe note is deleted or updated, the annotation should
	also be deleted or updated. Otherwise, we'll end up with annotations that aren't associated with any
	recipe note. */
		ON DELETE CASCADE
```

```
		ON UPDATE CASCADE,
	FOREIGN KEY (RecipeID) REFERENCES Recipe(RecipeID)
		/* Use CASCADE here because if the recipe is deleted or updated, the annotation should also
	be deleted or updated. Otherwise, we'll end up with annotations that aren't associated with any recipe.
	*/
		ON DELETE CASCADE
		ON UPDATE CASCADE
);

CREATE TABLE Receipt(
	ReceiptID UUID PRIMARY KEY,
	OCRResult VARCHAR,
	ImageFilePath VARCHAR UNIQUE NOT NULL,
	CreationDate TIMESTAMPTZ NOT NULL,
	AccountID UUID NOT NULL,
	FOREIGN KEY (AccountID) REFERENCES Account(AccountID)
		/* Use CASCADE here because if the account is deleted or updated, the receipt should also be
	deleted or updated. Otherwise, we'll end up with receipts that aren't associated with any user. */
		ON DELETE CASCADE
		ON UPDATE CASCADE
);

CREATE TABLE Inventory(
	InventoryID UUID PRIMARY KEY,
	Name VARCHAR NOT NULL,
	Description VARCHAR NOT NULL,
	Type VARCHAR NOT NULL
);

CREATE TABLE InventoryItem(
	InventoryItemID INT,
	InventoryID UUID,
	Quantity INT NOT NULL,
	CreationDate TIMESTAMPTZ NOT NULL,
	ExpirationDate TIMESTAMPTZ,
	IngredientName VARCHAR NOT NULL,
	ReceiptID UUID,
	PRIMARY KEY (InventoryItemID, InventoryID),
	FOREIGN KEY (InventoryID) REFERENCES Inventory(InventoryID)
		/* Use CASCADE here because if the inventory is deleted or updated, the inventory item should
	also be deleted or updated. Otherwise, we'll end up with inventory items that aren't associated with any
	inventory. */
		ON DELETE CASCADE
		ON UPDATE CASCADE,
	FOREIGN KEY (IngredientName) REFERENCES Ingredient(Name)
		/* Use NO ACTION here to prevent the ingredient from being deleted, because it doesn't make
	sense to delete an ingredient if there's an inventory item referencing it */
		ON DELETE NO ACTION
```

```
		/* Use CASCADE here because if the ingredient is updated, we should update it here to keep
	the reference correct */
		ON UPDATE CASCADE
		,
	FOREIGN KEY (ReceiptID) REFERENCES Receipt(ReceiptID)
		/* Use SET NULL here because if the receipt is being deleted, it seems that it is ok to just
	remove the reference here because inventory item doesn't have to have a receipt */
		ON DELETE SET NULL
		/* Use CASCADE here because if the receipt is updated, we should update it here to keep the
	reference correct */
		ON UPDATE CASCADE
);

/* Note: We couldn't force Inventory's total participation constraints in many-to-many relationships until we learn
assertions in the SQL unit */
CREATE TABLE AccountOwnsInventory(
	AccountID UUID,
	InventoryID UUID,
	PRIMARY KEY (AccountID, InventoryID),
	FOREIGN KEY (AccountID) REFERENCES Account(AccountID)
		/* Use CASCADE here because if the account is deleted, this relationship will no longer be
needed. */
		ON DELETE CASCADE
		/* Use CASCADE here because if the account is updated, we should update it here to keep the
	reference correct */
		ON UPDATE CASCADE,
	FOREIGN KEY (InventoryID) REFERENCES Inventory(InventoryID)
		/* Use CASCADE here because if the inventory is deleted, this relationship will no longer be
needed as well. */
		ON DELETE CASCADE
		/* Use CASCADE here because if the inventory is updated, we should update it here to keep the
	reference correct */
		ON UPDATE CASCADE
);
```

9. Use of AI tools

We didn't use any AI for this milestone